IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Received 30 September 2025, accepted 25 October 2025, date of publication 30 October 2025, date of current version 5 November 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3627277

APPLIED RESEARCH

A Retrieval-Augmented Generation Framework Based on a Knowledge Graph of Cybersecurity Vulnerabilities in Power Networks

XINGZHENG GAO[1,2] AND XING CHANG[1]

[1]Shenyang Institute of Computing Technology, Chinese Academy of Sciences, Shenyang 110168, China
[2]University of Chinese Academy of Sciences, Beijing 101408, China

Corresponding author: Xingzheng Gao (gaoxingzheng23@mails.ucas.ac.cn)

This work was supported by Shenyang Institute of Computing Technology, Chinese Academy of Sciences, Shenyang, China.

ABSTRACT To address the knowledge blind spots and hallucination issues of large language models (LLMs) in power grid cybersecurity, and to enhance their accuracy and practicality in decision support, this paper proposes a retrieval-augmented generation (RAG) framework based on a power grid security vulnerability knowledge graph. The framework integrates seven key steps—knowledge modeling, knowledge extraction, knowledge storage, vector indexing, intent recognition, similarity retrieval, and problem solving—to expand the contextual information of LLMs and enable enhanced generation. Experimental evaluation demonstrates the effectiveness of the proposed framework. Across six metrics—Answer Relevancy, Faithfulness, Context Precision, Context Recall, Answer Correctness, and Answer Similarity—the framework achieves scores of 92.41%, 91.06%, 98.17%, 97.28%, 94.78%, and 93.39%, respectively, outperforming existing RAG methods. In addition, performance tests show that, with relatively low generation time and token consumption, the framework achieves promising results in vulnerability verification and repair tasks, with verification and repair rates reaching 84% and 92%, respectively. Ablation studies further reveal that the problem-solving module has the greatest impact on answer relevance and fidelity, reducing them by 50.28 and 45.11 percentage points when removed. Similarly, the number of retrieved documents and similarity retrieval significantly affect contextual precision and recall, with the absence of similarity retrieval causing decreases of 18.17 and 22.28 percentage points. Qualitative assessments indicate that the framework performs well in mitigating hallucinations, handling global problem-solving, improving temporal adaptability, and enhancing interpretability, thereby demonstrating strong problem-solving capabilities. By integrating knowledge graphs with LLMs, this work not only offers a new approach for addressing practical issues but also contributes to the advancement of the power industry.

INDEX TERMS Power networks, large language models, knowledge graph, retrieval-augmented generation, prompt engineering.

I. INTRODUCTION

With the growing complexity of the international landscape and the rapid advancement of intelligent and information technologies, power systems have become prime targets for cyberattacks, making their security and stability a critical concern [1]. Current defense strategies often rely on constructing knowledge graphs, yet this approach faces several challenges in practice. First, building datasets typically demands extensive manual effort for data annotation. Second, constructing knowledge graphs requires essential steps such as entity recognition and relation extraction, while training the corresponding deep learning models is time-consuming and resource-intensive. Finally, although knowledge graph retrieval offers high accuracy, its results often lack interpretability.

In recent years, large language models (LLMs) have opened new avenues for research. LLMs, characterized

The associate editor coordinating the review of this manuscript and approving it for publication was Mueen Uddin.

© 2025 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

by massive parameter scales, are pretrained on extensive corpora to model semantic and probabilistic relationships in context, enabling them to understand and generate human language [2]. By directly querying LLMs, researchers can efficiently solve problems and significantly improve productivity. Moreover, their pretrained nature eliminates the need for manual dataset construction, thereby reducing labor costs.

However, applying LLMs to power system security poses three major limitations. First, because LLMs are primarily trained on general-purpose corpora, they are prone to hallucinations—generating content inconsistent with real-world facts—when addressing domain-specific issues. Second, the static nature of training data makes it difficult for LLMs to handle time-sensitive queries, such as identifying security vulnerabilities disclosed just a day earlier (e.g., a CVE identifier). Finally, although fine-tuning can partially mitigate these issues, it is costly, inefficient, and difficult to implement. These challenges highlight the limitations of relying solely on LLMs to address power system security problems.

Retrieval-Augmented Generation (RAG) [3] enhances large language models (LLMs) by retrieving relevant facts from a knowledge base and incorporating them into the generation process. This approach not only mitigates hallucinations but also improves timeliness, thereby enhancing the accuracy of model responses. Currently, mainstream RAG techniques are document-segmentation based. While effective in improving LLM outputs to some extent, this method has notable limitations.

First, document-segmentation-based RAG retrieves the most relevant text chunk using similarity search, integrates it with the query, and passes it to the LLM for generation. This works well when the user's question is localized within a specific text block, producing satisfactory answers. However, when the query requires understanding the broader context of the entire knowledge base—such as summarizing the central idea of a paper—the model performs poorly, as only the most relevant chunk is provided rather than the full context [4].

Second, the majority of data in this study on power system security is structured, with only a small portion consisting of unstructured text. Document segmentation disrupts the inherent relationships and integrity of structured data, making it more suitable for unstructured text. Since knowledge graphs organize information in a structured form, there is a pressing need to explore RAG methods that leverage knowledge graphs as the underlying knowledge base to address these challenges.

To address the above challenges, this paper proposes a Retrieval-Augmented Generation (RAG) framework based on a power network security vulnerability knowledge graph. The framework leverages large language models to construct the knowledge graph and uses it as the underlying knowledge base. It integrates seven key steps—knowledge modeling, knowledge extraction, knowledge storage, vector indexing, intent recognition, similarity retrieval, and problem solving—to generate solutions for power network security issues. Validation results demonstrate that the framework can be effectively applied in real-world power system scenarios, offering greater intelligence and efficiency compared to existing security protection approaches.

The main innovations and contributions of this paper are as follows:

- We propose a Retrieval-Augmented Generation (RAG) framework that uses a knowledge graph as the knowledge base. This approach effectively mitigates issues such as hallucinations, poor timeliness of generated results, incompatibility with structured data, and limited understanding of global queries, while improving the efficiency of problem-solving in practical applications.
- We employ large language models for knowledge graph construction and intent recognition, which significantly enhances construction efficiency, accelerates task execution, and reduces labor costs.
- The proposed framework can be deployed in real-world production environments. It enables more intelligent problem solving, reduces reliance on domain expertise, lowers deployment costs, and improves the overall quality of question answering.

II. RELATED WORK

A. RELATED RESEARCH IN POWER NETWORK SECURITY

In the field of power network security, knowledge graph technology has demonstrated remarkable effectiveness. A knowledge graph, composed of nodes and relationships, organizes information in a structured manner, enabling retrieval and reasoning based on semantic associations [5]. By collecting power network security data to construct knowledge graphs, researchers have developed powerful solutions for addressing cybersecurity challenges in power systems. For example, Liang et al. [6] conducted an in-depth study on key technologies for applying knowledge graphs to power network security, providing valuable insights for the field. Qiu et al. [7] proposed a graph-based method for quantifying cross-domain attack paths in cyber–physical power systems. By constructing hierarchical attack graphs and incorporating vertex vulnerabilities, edge weights, and network coupling relationships, they validated the method's effectiveness in evaluating the propagation risks of attack paths and their impact on physical grid operations, thereby addressing the challenge of cross-domain attack path construction and risk assessment. Similarly, Sun et al. [8] introduced a temporal knowledge graph method based on KNN and SVM. Their approach filtered noisy data by leveraging multi-K value curves from KNN to identify feature importance and applied SVM for supervised classification of four-dimensional data. The resulting dynamic temporal knowledge graph demonstrated adaptability to evolving environments, enhancing both the security and predictive accuracy of power monitoring systems.

Large language model (LLM) technology has expanded the range of solutions available for power network security. Since Google introduced the groundbreaking BERT model [9], both academia and industry have developed numerous advanced LLMs, such as OpenAI's GPT series [10], [11] and DeepSeek from DeepSeek Company [12], [13], [14], [15]. With their strong language understanding and generation capabilities, LLMs have been increasingly applied to address complex problems in power system security. For instance, Zaboli et al. [16] proposed a cybersecurity framework based on LLMs that integrates data preprocessing with human-in-the-loop (HITL) training. Their approach effectively detected anomalies in IEC 61850 communications and was validated through hardware-in-the-loop (HIL) testing, demonstrating the framework's efficacy in grid anomaly detection. Ibrahim and Kashef [17] presented a comprehensive methodology that included an overview of cybersecurity threats to smart grids, classification of attack surfaces and communication layers, and analysis of the core security triad—confidentiality, integrity, and availability—while also exploring the role of emerging technologies such as AI and LLMs in smart grid security. Similarly, Dalzell et al. [18] proposed an approach using LLMs to analyze SCADA alerts and construct a knowledge graph, addressing inefficiencies and decision delays faced by control room operators when managing large volumes of alerts. Experimental results showed that the method enables operators to rapidly extract key information through natural language queries, thereby improving decision-making speed and grid management efficiency. The study further demonstrated the potential of LLMs in knowledge base construction and SCADA alert data mining.

B. CURRENT RESEARCH ON RAG THEORY

Retrieval-Augmented Generation (RAG) combines information retrieval with natural language generation, leveraging external knowledge bases to help large language models (LLMs) produce more accurate and fact-supported responses [19]. RAG typically consists of four stages: indexing, retrieval, augmentation, and generation. In the indexing stage, texts from an external knowledge base are segmented into chunks, converted into vector representations, and stored in a vector database. During retrieval, a user query is embedded into a vector and compared against the database through similarity search to identify the most relevant text chunks. In the augmentation stage, the query and retrieved text chunks are incorporated into a predefined prompt template. Finally, in the generation stage, the augmented prompt is fed into an LLM to produce the desired output [20].

To further enhance the performance of Retrieval-Augmented Generation (RAG), many researchers have proposed novel frameworks and methods. For example, Nakano et al. [21] introduced WebGPT, which integrates a text-browsing environment with human feedback to optimize GPT-3's long-form question answering. On the ELI5 dataset, WebGPT produced answers of higher quality than both human demonstrations and top-voted Reddit responses, significantly improving the accuracy and reliability of open-domain QA. Asai et al. [22] proposed Self-RAG, a framework that strengthens factuality and generation quality through dynamic retrieval and self-reflective labeling, outperforming existing methods across multiple tasks. Rackauckas [23] developed RAG-Fusion, which combines retrieval-augmented generation with reverse fusion techniques to generate multi-perspective queries and reorder results, thereby improving accuracy and comprehensiveness, though at the cost of longer response times and the need for query relevance optimization. Yu et al. [24] introduced Auto-RAG, a self-directed RAG approach where LLMs autonomously plan retrieval, extract information, infer answers, and optimize queries in a multi-turn dialogue process. The model dynamically decides what and when to retrieve, generating the final answer only after sufficient external knowledge is obtained. Wei et al. [25] proposed InstructRAG, designed to improve accuracy and reliability in knowledge-intensive tasks through explicit denoising. By leveraging instruction-tuned LLMs, the framework automatically generates reasoning steps that explain how correct answers are derived from potentially noisy retrieval documents. Finally, Li et al. [26] presented a link-based RAG method that employs a two-level decomposition mechanism and dynamically retrieves historical cases for matching, thereby achieving hierarchical semantic decoupling and knowledge integration.

III. DATA

A. DATA SOURCES AND PREPROCESSING

The experimental data used in this study were obtained from the National Vulnerability Database of Information Security (CNVD) and the Security Reference platform. The latter is a high-end think tank focusing on the development and application of the cybersecurity industry, tracking international cybersecurity trends and providing high-quality intelligence for experts, scholars, and enterprises.

The CNVD dataset spans 2024–2025 and includes 1,167 reports, while the Security Reference dataset covers 2025 and contains 1,200 of the latest security incidents. These datasets primarily include fundamental information on power system security, such as vulnerability name, severity level, description, submission date, CNVD and CVE identifiers, discoverer, remediation plan, affected software versions, reference links, and vulnerability type.

The vulnerability reports are in structured XML format, whereas the security incident data are unstructured text. Structured data were parsed directly using code, while unstructured data were processed using large language models with prompt-engineering templates for knowledge extraction. During knowledge graph construction, data preprocessing included cleaning irrelevant, erroneous, and blank information to ensure consistency.

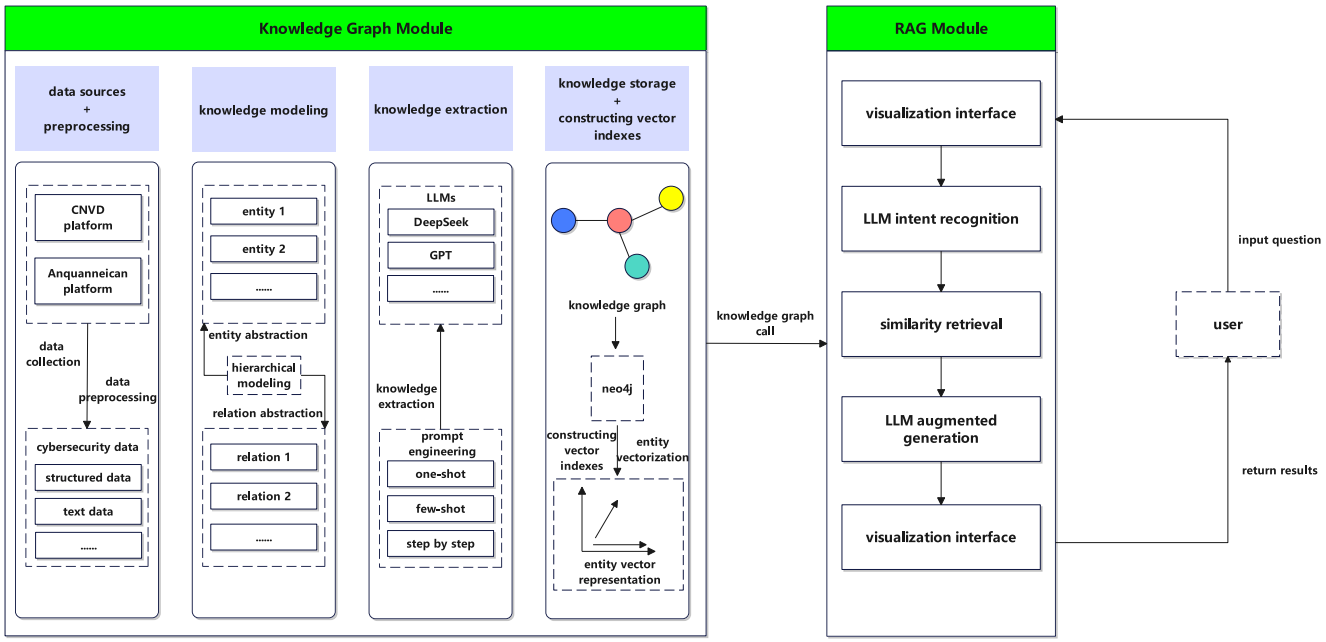


FIGURE 1. Overall schematic diagram of the framework.

B. CONSTRUCTION OF THE QA DATASET

To evaluate the framework's performance in practical applications, five domain experts were invited to annotate the collected data, resulting in the creation of a power network security QA dataset. The dataset contains 8,177 QA pairs, each consisting of six fields: QA ID, question type, question, generated answer, reference answer, and retrieved document fragments.

In this setup, the generated answer is the final response produced by the RAG framework, while the reference answer represents the expert-provided ground truth. The retrieved document fragments are the relevant text segments retrieved from the vector database for each query (set to 10 in this experiment). The question type distinguishes between simple, complex, and open-ended queries. Specifically, simple questions are factual queries with concise answers; complex questions require the model to integrate and summarize multiple retrieved fragments, typically resulting in longer responses; open-ended questions have no definitive answer and may elicit more extensive model-generated outputs.

IV. FRAMEWORK CONSTRUCTION

A. SYSTEM ARCHITECTURE

This study presents a newly designed framework, as illustrated in Figure 1. The operational process of the framework can be divided into seven steps: knowledge modeling, knowledge extraction, knowledge storage, vector index construction, intent recognition, similarity retrieval, and problem solving. Knowledge modeling focuses on abstracting power network security–related content into entities and relationships within a knowledge graph, thereby facilitating subsequent knowledge extraction. This is one of the core components of knowledge graph construction. Knowledge extraction leverages large language models (LLMs) in combination with prompt-based templates to identify entities and relationships in power security data. Compared to traditional methods of constructing knowledge graphs, this approach benefits from LLM's deep semantic understanding and expressive capabilities, significantly reducing the time needed to train dedicated deep learning models and improving construction efficiency. Knowledge storage involves saving extracted entities and relationships in a graph database, which supports future visualization and entity similarity retrieval tasks. To enable efficient and accurate entity retrieval, entity attributes must be transformed into vector representations. By establishing a similarity computation mechanism (e.g., cosine similarity) and assigning appropriate vector indexes to entities, retrieval efficiency can be greatly enhanced. The core task of intent recognition is to semantically classify user queries by using LLMs combined with prompt templates. Based on the identified intent categories, appropriate retrieval strategies are applied to achieve precise semantic matching. Using the constructed vector index, similarity retrieval is performed to identify the most relevant entity or top-k entities, depending on the type and specifics of the query. This facilitates the selection of suitable contextual information to be provided to LLM for enhanced generation. By integrating the retrieved context into the prompt template and passing it to the LLM for generation, the framework significantly improves the model's generation capability and enhances the efficiency of solving power network security problems.

B. KNOWLEDGE MODELING

The primary function of knowledge modeling is to abstract power network security–related data into entities and

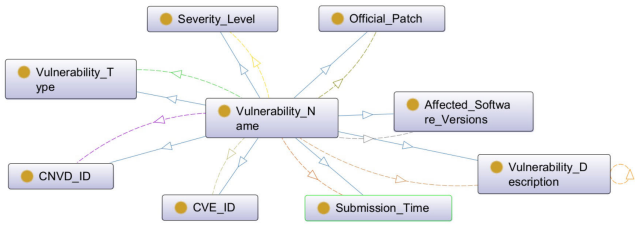


FIGURE 2. Knowledge modeling diagram.

TABLE 1. Entity name and description.

Entity Name	Description
Vulnerability Name	Standardized naming of vulnerabilities
CVE ID	The unique vulnerability ID from CVE
Submission Time	Vulnerability submission date
Vulnerability Description	A brief description of the vulnerability
CNVD ID	The unique vulnerability ID from CNVD
Severity Level	Severity level classification
Official Patch	Link to the patch released by the software manufacturer
Vulnerability Type	vulnerability category
Affected Software Versions	Affected software or system versions

TABLE 2. Relation name and description.

Start Entity	End Entity	Relation Name
Vulnerability Name	CVE ID	cveId
Vulnerability Name	CNVD ID	cnvdId
Vulnerability Name	Severity Level	vulSeverity
Vulnerability Name	Submission Time	vulSubmitTime
Vulnerability Name	Vulnerability Description	vulDetail
Vulnerability Name	Affected Software Versions	vulVersion
Vulnerability Name	Official Patch	vulRecover
Vulnerability Name	Vulnerability Type	vulType
Vulnerability Description	Vulnerability Description	next

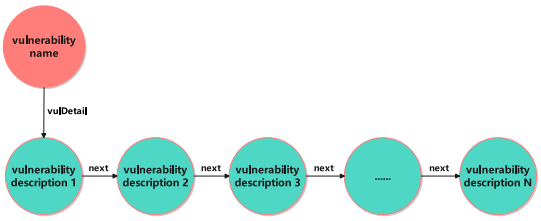


FIGURE 3. Schematic diagram of the structured representation.

relationships within a knowledge graph, thereby laying the foundation for subsequent knowledge extraction. A domain ontology for power network security vulnerabilities was constructed based on experimental data following the Stanford seven step methodology. Industry experts were invited to conduct rigorous manual annotations, resulting in the definition of nine entity types (e.g. vulnerability descriptions) and nine corresponding relationship types (e.g. cveId) to achieve structured knowledge representation. The final schema diagram, constructed using Protégé, is shown in Figure 2, and detailed information is provided in Tables 1 and 2.

Owing to the excessive length and complex technical details often found in vulnerability descriptions, an essential piece of information—personnel in the power security domain frequently needs to summarize and condense this

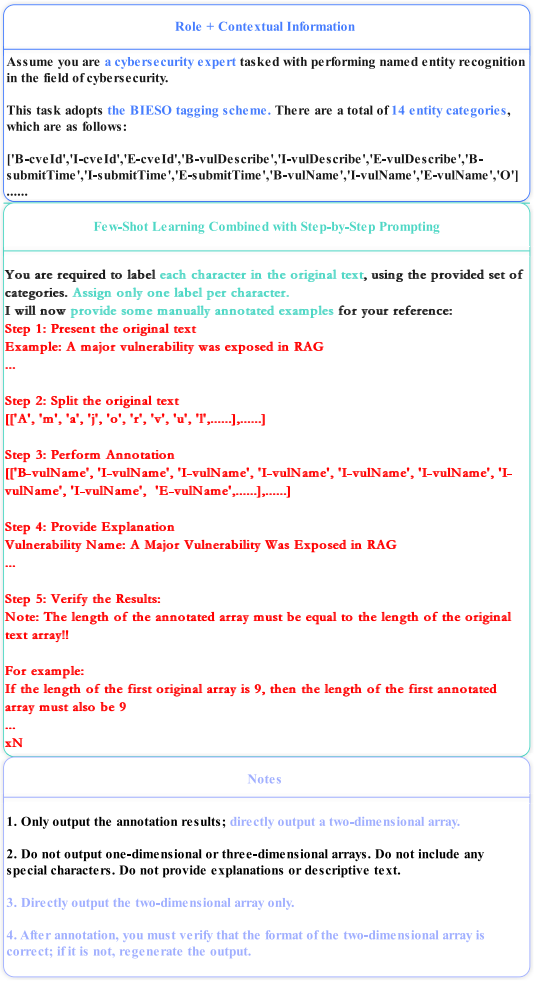


FIGURE 4. Schematic diagram of the few-shot combined with step-by-step prompt template.

information. This facilitates a quicker understanding of the overall vulnerability details, thereby improving problem-solving efficiency. Traditional Retrieval-Augmented Generation (RAG) approaches often struggle with global context comprehension. To address this issue, the proposed framework adopts an innovative structured representation method.

- **First**, the complete vulnerability description was divided into multiple text segments, each containing 500 characters.
- **Next**, each segment is stored in the knowledge graph as an independent entity.
- **Finally**, these entities are connected in a singly linked list structure through specific relationships, as illustrated in Figure 3.

This structured representation method organizes multiple context segments using entities and relationships. When a user raises a global-level question, the system first retrieves the most relevant entity via similarity search. Based on this entity, it matches and incorporates connected contextual entities, thereby expanding the model's input context. This approach provides an effective solution to the limitations

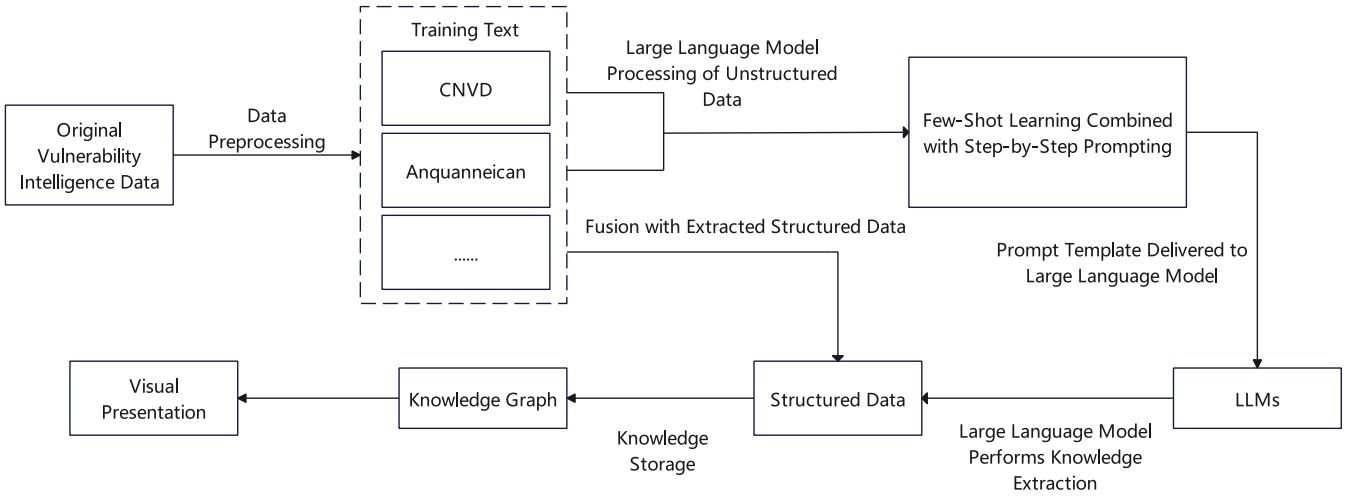


FIGURE 5. Knowledge graph construction process.

of insufficient global understanding of large language models.

C. KNOWLEDGE EXTRACTION

Knowledge extraction is a core component of the construction of knowledge graphs. It primarily involves extracting entities and relationships from power network security data based on prior knowledge modeling. Traditional deep learning–based methods often suffer from low efficiency and limited generalization in knowledge extraction tasks. In recent years, large language models (LLMs) have demonstrated strong language understanding and processing capabilities. To enhance both the accuracy and efficiency of the extraction, this study proposes an innovative approach that combines few-shot learning with step-by-step prompting to construct prompt templates. These templates are then passed to the LLM to perform knowledge extraction, significantly improving both the precision and efficiency. The detailed process is illustrated in Figure 4.

D. KNOWLEDGE STORAGE

Following the above process, a knowledge graph containing 5,763 entities and 16,100 relations was constructed and stored in the Neo4j graph database. As an industry-leading graph database, Neo4j is widely used in various application scenarios, including knowledge graph construction. It represents graph structures through entities, relationships, properties, and labels, and offers several core advantages. First, its Cypher query language, which is syntactically similar to a Structured Query Language (SQL), supports efficient operations for creating, reading, updating, and deleting graph data. Second, Neo4j provides an intuitive visual interface that allows users to directly observe and manipulate graph data relationships. Finally, Neo4j supports the integration of various plug-ins to extend its capabilities. For example, the APOC plug-in offers advanced graph algorithms and



FIGURE 6. Partial schematic diagram of the knowledge graph.

data science functionalities, such as community detection algorithms and vector similarity retrieval. These features enable more advanced applications of the graph data. Figure 5 illustrates the complete construction process, while a portion of the resulting graph stored in Neo4j is shown in Figure 6.

E. CONSTRUCTING VECTOR INDEXES

To enable efficient entity retrieval, Neo4j provides a vector indexing mechanism. An example of a Cypher statement for adding a vector index to vulnerability name entities is shown in Figure 7.

In this example, the vector index is named vul_name _embeddings, and the transformed vectors are stored in

```
CREATE VECTOR INDEX vul_name_embeddings IF NOT EXISTS
FOR (m:Vulnerability_Name) ON (m.vulNameEmbedding)
OPTIONS { indexConfig: {
   `vector.dimensions`: 3072,
   `vector.similarity_function`: 'cosine'
}}
```

FIGURE 7. Illustrative cypher statement for adding a vector index to vulnerability name entities.

```
MATCH (m:Vulnerability_Name) WHERE m.vul_name IS NOT NULL
      WITH m, genai.vector.encode(
         m.vul_name,
         "OpenAI",
         {
           token: openAiApiKey,
           model: openAiModel
         }) AS vector
CALL db.create.setNodeVectorProperty(m, "vulNameEmbedding", vector)
```

FIGURE 8. Illustrative cypher statement for converting entity attributes into vector representations.

the vulNameEmbedding attribute of the entity. The value 3072 indicates the dimensionality of the vector representation of the entity attribute, while cosine specifies that the cosine similarity function is used to match entities during graph retrieval. The formula for computing cosine similarity is given in (1).

$$\cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\|\|\mathbf{B}\|} = \frac{\sum_{i=1}^{n} A_i B_i}{\sqrt{\sum_{i=1}^{n} A_i^2}\sqrt{\sum_{i=1}^{n} B_i^2}} \tag{1}$$

In this context, A and B represent n-dimensional vectors. In text retrieval, cosine similarity is a commonly used similarity measurement method. It assesses the similarity between text vectors by calculating the cosine of the angle between them, effectively ignoring differences in length and enabling efficient evaluation of content relevance.

After constructing the vector index, the next step is to use an embedding model to convert entity attributes into vector representations. In Natural Language Processing (NLP), embeddings transform text into vectors, enabling computers to understand and process textual data for use in various NLP tasks. An example of a Cypher statement is shown in Figure 8.

In this context, vul_name represents an entity attribute to be converted into a vector, specifically referring to the vulnerability name. The OpenAI embedding model is used for this process, where token denotes the API key for accessing the embedding model, model specifies the name of the embedding model, and vulNameEmbedding attribute stores the resulting vector representation.

F. INTENT RECOGNITION

In the field of intelligent question answering, intent recognition primarily classifies user queries and applies different processing strategies based on the identified question category. In this experiment, five question categories were defined, with detailed descriptions provided in Table 3.

TABLE 3. Question categories and descriptions.

ID	Question Category
1	Ask for vulnerability details based on the vulnerability name
2	Ask for remediation solutions based on the vulnerability name
3	Ask for vulnerability names based on the software name
4	Ask for vulnerability names based on the CVE identifier
5	Ask for vulnerability names based on the submission date

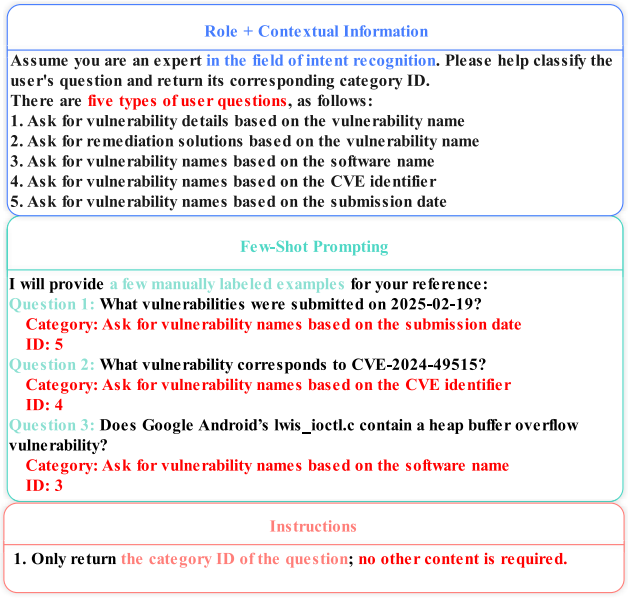


FIGURE 9. Schematic diagram of the prompt template for intent recognition.

Intent recognition is a multi-class classification task. To reduce the time required to train dedicated deep learning models and improve the efficiency of intent recognition, this study adopts an approach that combines large language models with prompt-based templates. The template used for this purpose is illustrated in Figure 9.

G. SIMILARITY RETRIEVAL

After constructing vector indexes for the entity attributes, we perform similarity retrieval between the user's input question and the indexed entities based on the identified question type. The system returns the most relevant or top-k entities, enabling the corresponding contextual information to be passed to the large language model for enhanced generation. An example of a Cypher query is shown in Figure 10.

Here, the question represents the user's input query, the token denotes the OpenAI API key, and the model specifies the embedding model used. The vul_name_embeddings refers to the name of the vector index, top_k indicates the number of top-matching entities to return, result_node represents the retrieval result, and score denotes the similarity score.

```
WITH genai.vector.encode(
        question,
        "OpenAI",
        {
          token: openAiApiKey,
          model: openAiModel
        }) AS question_embedding
CALL db.index.vector.queryNodes(
        'vul_name_embeddings',
        top_k,
        question_embedding
        ) YIELD node AS result_node, score
RETURN result_node.vul_name, score
```

FIGURE 10. Illustrative cypher statement for similarity retrieval based on user query.

```
MATCH (p)-[:`cveId`]->(node),(p)-[:`vulDetail`]->(q)
RETURN
"Vulnerability Name:" + p.vul_name + "\n" +
"CVE ID:" + node.cve_id + "\n" +
"Vulnerability Description:" + q.vul_detail AS text, score,
{foo:"bar",source:""} AS metadata
```

FIGURE 11. Illustrative cypher statement for retrieving contextual information of entities based on similarity search.

H. PROBLEM SOLVING

Based on the different question categories, the corresponding Cypher query statements were constructed. After retrieving relevant knowledge from the knowledge graph through a similarity search, the retrieved content is embedded into a prompt template to expand the model's context, and then passed to the large language model for enhanced generation. All the above processes were implemented using the LangChain framework. An example of a Cypher query is shown in Figure 11, and the prompt template for problem solving is illustrated in Figure 12.

In this context, node represents the node retrieved via similarity search using the vector index, and the score indicates the similarity score. The relationship between p and node is defined as cveId, and that between p and q is defined as vulDetail. After matching these three entities p, node, and q based on their relationships, the vulnerability name, CVE ID, and vulnerability description of these entities are returned, respectively, for the purpose of enriching the context fed into the large language model.

The complete execution process of the proposed framework is represented below in Algorithm 1.

V. EXPERIMENTS AND ANALYSIS

A. EXPERIMENTAL DESCRIPTION

This study designed and conducted four experiments. The first compared the proposed framework with existing RAG methods across six metrics—Answer Relevancy, Faithfulness, Context Precision and Recall, Answer Correctness, and Answer Similarity—to comprehensively evaluate overall performance. The second experiment assessed practical effectiveness using four indicators: generation time, token consumption, vulnerability verification rate, and vulnerability repair rate. To further examine the contribution of each

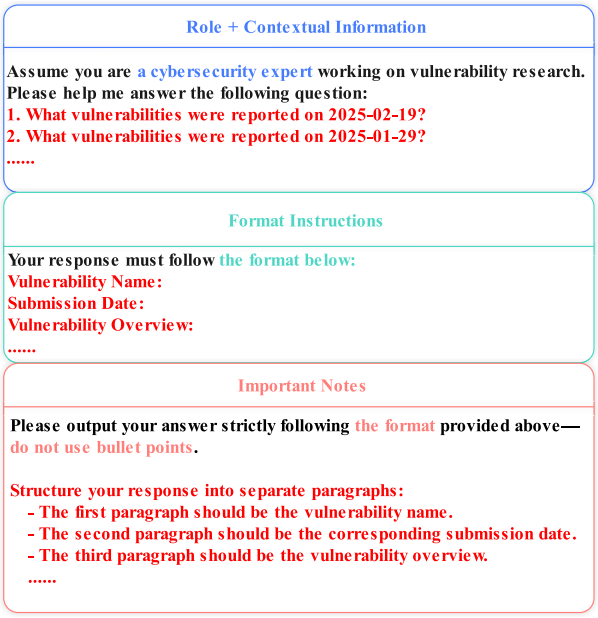


FIGURE 12. Schematic diagram of the prompt template for problem solving.

```
MATCH window=(:`Vulnerability Description`)-[:next*0..]->(node)-
[:next*0..]->(:`Vulnerability Description`)
WITH node, score, window as longestWindow
        ORDER BY length(window) DESC LIMIT 1
      WITH nodes(longestWindow) as chunkList, node, score
        UNWIND chunkList as chunkRows
      WITH collect(chunkRows.vul_detail) as textList, node, score
RETURN apoc.text.join(textList, " \n ") as text,score,
{foo:"bar",source:""} AS metadata
```

FIGURE 13. Illustrative cypher statement for context expansion based on matched entity paths.

module, an ablation study was conducted by sequentially removing the problem-solving prompt template, the similarity retrieval module, and by adjusting the number of retrieved documents, thereby measuring their specific impact on system performance. The fourth experiment provided a qualitative analysis along four dimensions: mitigation of model hallucination, improvement of interpretability, enhancement of timeliness, and capacity for solving global problems. For global problem-solving, the framework was evaluated using vulnerability description entities to verify the model's limitations in global understanding. An example Cypher query is shown in Figure 13, and the overall Q&A workflow is illustrated in Figure 14.

The primary function of the Cypher query is as follows: based on the user's input question, it first retrieves the most relevant vulnerability description entity through similarity matching. It then traces zero or more next relationships from this entity to identify its preceding and subsequent contextual entities, selecting the longest path to maximize contextual coverage. The attributes (vul_detail) of all entities along this path are extracted, concatenated with line breaks, and

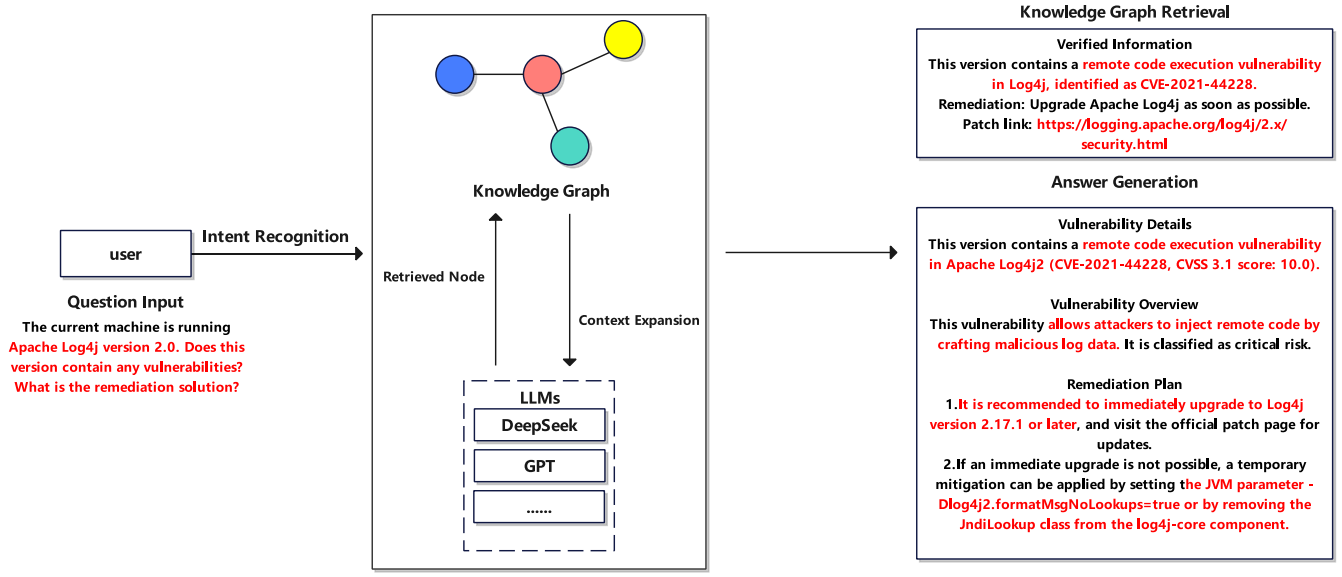


FIGURE 14. Overall question-answering workflow of the framework.

provided to the large language model to enrich the contextual information.

B. EXPERIMENTAL ENVIRONMENT AND BASELINE MODELS

The experimental environment was configured as follows: operating system, Windows 10; memory, 16 GB; Python version, 3.12; Neo4j version, neo4j-community-5.26.4; text embedding model, text-embedding-3-large; generation model, deepseek-v3; evaluation framework, RAGAS [27]; evaluation large language model, GLM-4-Flash; evaluation embedding model, Embedding-3; and token calculation tool, tiktoken.

The baseline RAG configuration was as follows: FAISS served as the vector database with cosine similarity for retrieval. Documents were segmented into 500-character chunks with a 50-character overlap. Both the embedding and generation models were consistent with those used in the proposed framework, and the overall process was implemented via the LangChain framework.

C. EVALUATION METRICS

This study employs ten evaluation metrics: Answer Relevancy, Faithfulness, Context Precision, Context Recall, Answer Correctness, Answer Similarity, Vulnerability Verification Rate, Vulnerability Repair Rate, Token Consumption, and Generation Time.

Answer Relevancy measures the degree of alignment between the model's output and the input question, ranging from 0 to 1, with higher values indicating stronger relevance. Specifically, the evaluation model infers a possible question from the generated answer and computes its similarity with the original question (Equation 2). Here, N denotes the number of questions, Vgi the embedding vector of the inferred question, Vo the embedding vector of the original question, and cos the cosine similarity function.

$$\text{Answer Relevancy} = \frac{1}{N}\sum_{i=1}^{N}\cos(V_{gi}, V_o) \tag{2}$$

Faithfulness assesses the credibility of generated answers by determining how many claims can be directly supported by the retrieved context. Scores range from 0 to 1, with higher values indicating greater factual accuracy and lower values reflecting hallucinations [28]. Its formulas are shown in Equations (3) and (4), where g is the generated output, N the number of claims, ai the i-th claim, c the context, and infer(ai,c) an indicator of whether claim ai can be inferred from c.

$$\text{Faithfulness} = \frac{1}{N}\sum_{i=1}^{N}\text{infer}(a_i, c) \tag{3}$$

$$\text{infer}(a_i, c) = \begin{cases} 1, & \text{if } a_i \text{ can be inferred from } c \\ 0, & \text{otherwise} \end{cases} \tag{4}$$

Context Precision evaluates whether relevant chunks are ranked above irrelevant ones. In other words, it measures the extent to which relevant chunks appear at the top of the retrieved results (Equations 5–6). Here, K is the total number of retrieved documents, Precision@k is the ratio of relevant chunks within the top k, and vk is a relevance indicator (1 if relevant, 0 otherwise).

$$\text{Context Precision@K} = \frac{\sum_{k=1}^{K}(\text{Precision@k} \times v_k)}{\text{Total number of relevant items in the top K results}} \tag{5}$$

TABLE 4. Comprehensive experimental results.

Method	Answer Relevancy	Faithfulness	Context Precision	Context Recall	Answer Correctness	Answer Similarity
SimpleRAG	0.4316	0.7	0.6744	0.7863	0.6325	0.63
SelfRAG	0.6747	0.6937	0.7712	0.7933	0.73	0.7112
AutoRAG	0.7852	0.7833	0.8715	0.85	0.8096	0.8161
LinkedRAG	0.8522	0.8538	0.9427	0.9221	**0.9532**	0.9328
InstructRAG	0.8893	0.8527	0.9512	0.9338	0.9409	0.9335
MyMethod	**0.9241**	**0.9106**	**0.9817**	**0.9728**	0.9478	**0.9339**

TABLE 5. Actual application performance evaluation results.

Method	Tokens /Request	Generation Time (s)/Request	Vulnerability Validation Rate	Vulnerability Repair Rate
SimpleRAG	**225**	**10.7**	20%	12%
SelfRAG	337	58.8	60%	76%
AutoRAG	737	50.4	68%	72%
LinkedRAG	451	30.4	80%	88%
InstructRAG	672	44.7	76%	88%
MyMethod	464	23.3	**84%**	**92%**

TABLE 6. Ablation experiment results.

Ablation Experiment	Answer Relevancy	Faithfulness	Context Precision	Context Recall	Answer Correctness	Answer Similarity
Complete Model	0.9241	0.9106	0.9817	0.9728	0.9478	0.9339
Abl-1	0.4213	0.4595	0.9817	0.9728	0.6728	0.6260
Abl-2	0.3826	0.4068	0.80	0.75	0.3848	0.5324
Abl-3	0.3256	0.3229	0.7273	0.6818	0.3307	0.5187

$$\text{Precision@k} = \frac{\text{true positives@k}}{\text{true positives@k} + \text{false positives@k}} \tag{6}$$

Context Recall measures the proportion of relevant documents successfully retrieved. More concretely, it checks whether each sentence in the reference answer(GT) appears in the retrieved context (Equation 7).

$$\text{Context Recall} = \frac{|\text{GT claims that can be attributed to context}|}{|\text{Number of claims in GT}|} \tag{7}$$

Answer Similarity quantifies the semantic similarity between the generated answer and the reference answer, where Vanswer and Vreference denote their embedding vectors, and cos represents the cosine similarity function.

$$\text{Answer Similarity} = \cos(V_{answer}, V_{reference}) \tag{8}$$

Answer Correctness evaluates the factual accuracy and validity of the generated answer relative to the reference.

Generation Time measures the total time taken from when the user inputs a query until the model outputs the result.

Vulnerability Verification Rate is the ratio of verifiable vulnerabilities to the total number of vulnerabilities. A vulnerability is considered verifiable if practitioners can confirm its existence through model-generated outputs (e.g., validation steps or proof of concept). The testing environment is simulated using virtual machines and online vulnerability platforms.

TABLE 7. Qualitative assessment 1 question list.

Question ID	Question Details
1	What are the details of the CSRF in JFinalCMS?
2	What is the remediation for the CSRF in JFinalCMS?
3	Does Android have a heap buffer overflow vulnerability?
4	What vulnerability corresponds to CVE-2024-49515?
5	What vulnerabilities were reported on 2025.02.19?

$$\text{Vulnerability Verification Rate} = \frac{\text{Number of verifiable vulnerabilities}}{\text{Total number of vulnerabilities}} \tag{9}$$

Vulnerability Repair Rate is the ratio of vulnerabilities with available fixes to the total number of vulnerabilities. A vulnerability is considered repairable if the model provides an official patch link.

$$\text{Vulnerability Repair Rate} = \frac{\text{Number of vulnerabilities with available fixes}}{\text{Total number of vulnerabilities}} \tag{10}$$

Token Consumption measures the total number of tokens used during model execution, as defined by its formula.

$$\text{Token Consumption} = \text{Number of input tokens} + \text{Number of output tokens} \tag{11}$$

TABLE 8. Summary of comparison results 1.

ID	Knowledge Graph	Large Language Model	Framework-Enhanced Generation
1	Provided the affected versions of JFinalCMS and the underlying cause of the vulnerability, but lacked detail.	Did not provide specific versions or technical details due to lack of context.	Provided both the affected versions and root cause, along with detailed explanations tailored to the software and vulnerability.
2	Did not offer a concrete remediation plan.	Offered only a general fix for CSRF vulnerabilities, without specifics for the JFinalCMS case.	Provided a detailed fix including a proof of concept (PoC) and a link to the vendor's homepage, based on the specific mechanism of the JFinalCMS CSRF vulnerability.
3	Presented the CVE, CNVD identifiers, and the vulnerability mechanism.	Did not return any information due to lack of timeliness.	Provided the identifiers, the root cause, and the resulting impact of the vulnerability.
4	Provided the identifier and a brief description of the impact.	Did not return any information due to lack of timeliness.	Provided the identifier, analyzed the cause, and expanded on the impact of the vulnerability.
5	Provided information on two vulnerabilities and outlined their impact.	Did not return any information due to lack of timeliness.	Provided the submission date, analyzed the cause, and expanded on the impact of the vulnerabilities.

TABLE 9. Qualitative assessment 2 question list.

Question ID	Question Description	Question Type
1	What is Retrieval-Augmented Generation (RAG)?	Local Question
2	What is the major vulnerability exposed in RAG?	Global Question
3	What is crosstalk in quantum computing?	Local Question
4	What do vulnerabilities in quantum computing primarily refer to?	Global Question

TABLE 10. Summary of comparison results 2.

ID	RAG + LLM	Framework-Enhanced Generation
1	Similar meaning expressed	Similar meaning expressed
2	Only states that the major vulnerability in RAG is model alignment issues, without explaining the underlying mechanism.	Provides a further explanation of the alignment vulnerability in RAG, including its root cause.
3	Similar meaning expressed	Similar meaning expressed
4	Only states that quantum computing vulnerabilities involve risks of intrusion using classical hacking techniques (e.g., Rowhammer attacks), without elaborating on the underlying principles.	Not only introduces quantum computing vulnerabilities, but also explains the specific technical principles behind them.

D. COMPREHENSIVE EXPERIMENTS

To evaluate overall performance, 30% of the power network security QA dataset was randomly selected for testing. We compared the proposed framework with the baseline RAG method, InstructRAG, LinkedRAG, AutoRAG, and SelfRAG.

The experimental results are summarized in Table 4, with the best scores highlighted in bold. As shown, compared with InstructRAG, our framework achieved improvements of 3.48% in Answer Relevancy, 5.79% in Faithfulness, 3.05% in Context Precision, 3.90% in context Recall, 0.69% in Answer Correctness, and 0.04% in Answer Similarity. Relative to the baseline RAG, the improvements were more substantial: 49.25%, 21.06%, 30.73%, 18.65%, 31.53%, and 30.39%, respectively—demonstrating the clear advantages of our approach.

These gains are primarily due to the limitations of document-splitting strategies in traditional RAG methods. When structured data are segmented into multiple chunks, contextual integrity is compromised. For example, key–value pairs from a single structured record may be divided across different chunks. If a user's query spans multiple chunks, the retrieval system may only return part of the relevant information, reducing retrieval accuracy and preventing the language model from accessing complete context. This often leads to incomplete reasoning and incorrect answers, whereas our framework preserves contextual integrity and mitigates this issue.

E. APPLICATION PERFORMANCE EVALUATION

To assess the practical performance of the proposed framework, we randomly selected 50 queries from the QA dataset,

Algorithm 1 Overall Execution Process of the Framework

```
Set model API key, model name, and corresponding prompt templates.
Collect and preprocess data.
Call the large language model with prompt templates to perform knowledge extraction.
Store the extracted knowledge graph in a Neo4j graph database.
Build vector indexes and convert entities into vector representations using an embedding model.
Define the user input question.
if Assume the user question falls under category 1 then
    Define a custom Cypher query for retrieving relevant information from the knowledge graph.
    Use LangChain to construct a vector database based on the vector index and custom Cypher query.
    Build a retriever from the vector database for similarity-based retrieval.
    Construct a QA chain based on the retriever.
    Feed the context retrieved via the QA chain into the large language model for enhanced generation.
    Return the output generated by the large language model.
else if Assume the user question falls under category 2 then
    ......
else if ..... then
    ......
else if Assume the user question falls under category N then
    ......
end if
```

including 25 vulnerability verification queries and 25 vulnerability repair queries. We evaluated the average token consumption, generation time, vulnerability verification rate, and vulnerability repair rate to measure applicability in real-world environments.

The experimental results are summarized in Table 5, with the best values highlighted in bold. The observations are as follows:

Baseline RAG achieved the lowest token consumption and fastest generation time, indicating the highest efficiency. However, its vulnerability verification and repair rates were extremely low, suggesting poor performance in handling complex vulnerability-related tasks.

SelfRAG and AutoRAG improved verification and repair accuracy but incurred higher generation times. AutoRAG also had the highest token consumption, reducing efficiency and making both approaches less suitable for real-time applications.

LinkedRAG and InstructRAG achieved relatively strong verification and repair rates but required more tokens and longer response times.

Our framework achieved the best overall results, with the highest verification (84%) and repair rates (92%), while maintaining relatively short generation time (23.3 seconds) and moderate token consumption (464). This advantage stems from the use of vector indexing, which reduces retrieval latency. The results demonstrate that our framework achieves an optimal balance between effectiveness and efficiency, delivering both rapid response and high accuracy.

F. ABLATION EXPERIMENTS

To evaluate the contribution of each component in our framework, three ablation experiments were conducted, denoted as Abl-1, Abl-2, and Abl-3. The experiments involved sequentially removing: (1) the problem-solving prompt template, (2) the similarity-based retrieval (replaced with exact matching), and (3) the Top-k recall mechanism (returning only the single most relevant block), while observing performance changes.

The results, shown in Table 6, indicate the following:

Abl-1 (removing the prompt template) had a decisive impact on generation performance. Answer Relevancy, Faithfulness, Answer Correctness, and Answer Similarity dropped by 50.28%, 45.11%, 27.5%, and 30.79%, respectively.

Abl-2 (disabling similarity-based retrieval) reduced retrieval performance, with corresponding decreases of 18.17% and 22.28%. Lower retrieval quality also degraded model context, further impacting Answer Relevancy, Faithfulness, Answer Correctness, and Answer Similarity.

Abl-3 (removing the Top-k recall mechanism) reduced the number of retrieved text blocks, leading to additional declines across all evaluation metrics.

G. QUALITATIVE ASSESSMENTS

Based on the classification of questions through intent recognition, experts from the electric power and cybersecurity domains were invited to conduct specialized discussions. Considering typical application scenarios in the field of power system security, five representative questions were selected, as shown in Table 7. These are primarily used for a comparative analysis of the output results generated by three different approaches: knowledge graph retrieval, large language model generation, and enhanced generation using the proposed framework. The results of the comparision are presented in Appendix A, and a summary of the comparison is presented in Table 8.

The experimental results demonstrate that applying the proposed framework for enhanced generation with large language models in the field of power network security can provide users with clear and comprehensive solutions to security issues, thereby significantly improving the efficiency of professionals in handling power network security incidents.

For example, regarding Question 1 (What are the details of the CSRF in the JFinalCMS?), although the results retrieved from the knowledge graph are highly accurate, they lack clarity and completeness, with limited interpretability. The

Knowledge Graph Retrieval Result	JFinalCMS is a content management system. **Version 1.0 of JFinalCMS contains a Cross-Site Request Forgery (CSRF) vulnerability, which arises from the failure to properly verify whether requests to the file /admin/tag/save originate from trusted users**. An attacker could exploit this flaw to craft malicious requests that deceive victims into performing sensitive operations.
LLM	**Analysis of the CSRF Vulnerability in JFinalCMS** JFinalCMS is a content management system built on the JFinal framework. The details of its CSRF vulnerability are as follows: **Specific Manifestation in JFinalCMS** **According to public vulnerability reports, certain versions of JFinalCMS** suffer from the following CSRF-related issues: **1.** Lack of CSRF Protection Mechanisms: Critical operations (e.g., administrative functions) lack effective CSRF token validation. **2.** Sensitive Operations Can Be Forged: Attackers can construct malicious pages to trick administrators into performing unintended actions. **3.** Scope of Impact: This may lead to privilege escalation, data tampering, and other security risks. **For more precise vulnerability details (e.g., specific affected version numbers), it is recommended to consult the National Vulnerability Database (CNNVD) or relevant security advisories.**
Framework-Enhanced Generation	**Vulnerability Name: JFinalCMS Cross-Site Request Forgery Vulnerability** JFinalCMS is a Java-based Content Management System (CMS) widely used in enterprise websites, news portals, blogging platforms, and similar applications. It offers rich modules such as article management, user permission control, and customizable templates, enabling users to quickly build and manage website content. **The main consequence of this vulnerability is that an attacker could perform unintended actions on behalf of the victim without their knowledge.** **Specifically, after a victim logs into the JFinalCMS backend system, the attacker can craft a malicious link or page to lure the victim into clicking it.** **Once the link is accessed, the forged request is sent to the backend interface of JFinalCMS—such as the save function in the tag management module (/admin/tag/save).** **The root cause lies in the fact that JFinalCMS version 1.0 did not implement effective CSRF protection when designing the /admin/tag/save interface.** **Normally, the system should enforce request validity by using random tokens (CSRF Tokens) or verifying the HTTP Referer header.** However, this interface in JFinalCMS lacks adequate request validation, allowing attackers to bypass authentication and trigger sensitive actions via forged HTTP requests.

FIGURE 15. Comparison results for question 1.

Knowledge Graph Retrieval Result	**The vendor has not yet provided a vulnerability fix.** Please monitor the vendor's homepage for updates: https://github.com/hadagaga/vuln/blob/master/JFinalCMS/Cross_Site_Request_Forgery/Cross-Site-Request-Forgery.md
LLM	**Fix Recommendation for the JFinalCMS Cross-Site Request Forgery (CSRF) Vulnerability** To address the CSRF vulnerability in JFinalCMS, the following remediation measures are recommended: 1. Add CSRF Tokens to Forms 2. Handle CSRF Tokens in AJAX Requests 3. Ensure all forms and AJAX requests include CSRF tokens. 4. Tokens should be one-time use or have an expiration time. 5. Use POST requests instead of GET for critical operations. 6. Regularly update the framework to apply the latest security patches.

Framework-Enhanced Generation	**Vulnerability Name: JFinalCMS Cross-Site Request Forgery (CSRF) Vulnerability**
	Remediation Plan:
	The CSRF vulnerability in JFinalCMS version 1.0 stems from the failure of the /admin/tag/save endpoint to adequately verify the legitimacy of request origins.
	The core approach to fixing this vulnerability is to implement a CSRF protection mechanism.
	Below is a simplified PoC (Proof of Concept) demonstrating how to defend against CSRF attacks through token generation and verification:
	``java
	// 1. Server-side generation and storage of CSRF token in session
	public void generateCSRFToken(HttpServletRequest request) {
	String token = UUID.randomUUID().toString();
	request.getSession().setAttribute("CSRF_TOKEN", token);
	}
	// 2. Embedding the token into the HTML form (front-end example)
	<form action="/admin/tag/save" method="POST">
	<input type="hidden" name="csrfToken" value="${sessionScope.CSRF_TOKEN}">
	<input type="submit" value="save label">
	</form>
	// 3. Server-side verification of the token (interceptor example)
	public class CSRFInterceptor implements Interceptor {
	@Override
	public void intercept(Invocation inv) {
	HttpServletRequest request = inv.getController().getRequest();
	String sessionToken = (String) request.getSession().getAttribute("CSRF_TOKEN");
	String requestToken = request.getParameter("csrfToken");
	if (sessionToken == null || !sessionToken.equals(requestToken)) {
	inv.getController().renderError(403, "Error");
	return;
	}
	inv.invoke();
	}
	}
	``
	Reference Link / Vendor Homepage:
	https://github.com/hadagaga/vuln/blob/master/JFinalCMS/Cross_Site_Request_Forgery/Cross-Site-Request-Forgery.md

FIGURE 16. Comparison results for question 2.

Knowledge Graph Retrieval Result	**Vulnerability Name: Google Android lwis_ioctl.c Heap Buffer Overflow Vulnerability** **CVE ID: CVE-2024-47032** **CNVD ID: CNVD-2025-01824** **Submission Date: January 10, 2025** **Google Android is an open-source operating system developed by Google, based on the Linux kernel.** **A security vulnerability has been identified in Google Android, caused by a heap buffer overflow in the construct_transaction_from_cmd function within the lwis_ioctl.c file. This flaw may lead to out-of-bounds writes. At present, detailed technical information about this vulnerability has not been disclosed.**
LLM	**Currently, there is no publicly available authoritative vulnerability report**—such as from CVE, the Android Security Bulletin, or Google Project Zero—**that explicitly confirms a heap buffer overflow vulnerability in the lwis_ioctl.c file.**
Framework-Enhanced Generation	**Vulnerability Name: Google Android lwis_ioctl.c Heap Buffer Overflow Vulnerability** **CVE ID: CVE-2024-47032** **CNVD ID: CNVD-2025-01824** **Submission Date: January 10, 2025** **Vulnerability Overview:** **This vulnerability exists in the lwis_ioctl.c file of the Google Android operating system and specifically affects the construct_transaction_from_cmd function. It is classified as a heap buffer overflow vulnerability, which may allow attackers to perform out-of-bounds memory writes. This could result in memory corruption or arbitrary code execution.**Buffer overflow vulnerabilities generally occur when a program writes data to a buffer without performing adequate size checks, leading to data being written beyond the buffer's allocated boundary. This behavior may overwrite critical adjacent memory, alter program execution flow, or even allow execution of malicious code. **In Android systems, such vulnerabilities could be exploited to escalate privileges or bypass security mechanisms.** Google typically addresses these kinds of vulnerabilities through monthly security patches. Users are strongly advised to keep their systems up to date to ensure timely protection against such threats.

FIGURE 17. Comparison results for question 3.

Knowledge Graph Retrieval Result	**Vulnerability Name: Adobe Substance 3D Painter Untrusted Search Path Vulnerability** **CVE ID: CVE-2024-49515** **Vulnerability Overview:** Adobe Substance 3D Painter is a 3D texture processing application developed by Adobe Inc. in the US. **Adobe Substance 3D Painter has a security vulnerability that can be exploited by attackers to execute arbitrary code.**
LLM	**As of my latest knowledge (July 2024), CVE-2024-49515 has not been widely documented in public vulnerability databases (such as NVD, MITRE, etc.), so detailed information about the vulnerability is not yet available.**
Framework-Enhanced Generation	**Vulnerability Name:** The Adobe Substance 3D Painter untrusted search path vulnerability is a serious security flaw found in the Substance 3D Painter application developed by Adobe. This application is a professional tool widely used in 3D texture painting and material design, relied upon by many game developers and digital artists. **The core issue of this vulnerability lies in the application's failure to properly validate or restrict the trustworthiness of search paths when loading external resources or performing specific operations, allowing attackers to use carefully crafted malicious files or path hijacking techniques to trick the application into executing unauthorized code.** **CVE ID:** **CVE-2024-49515** **Vulnerability Overview:** Adobe Substance 3D Painter is a powerful 3D texture painting tool widely used in game development, visual effects, and digital art creation. **However, the existence of the CVE-2024-49515 vulnerability allows attackers to exploit flaws in how the application loads dynamic link libraries (DLLs) or other external resources to inject malicious code into the application's process.** **Specifically, when Substance 3D Painter attempts to load certain system or user-provided library files at runtime, if it does not properly restrict the search path or verify the integrity of the files, an attacker can place malicious DLL files in specific directories to fully control the application's execution flow.** This attack method may result in arbitrary code execution, enabling attackers to steal sensitive data, compromise system integrity, or even further penetrate the entire network environment. For professionals and enterprises using Adobe Substance 3D Painter, the potential harm of this vulnerability is extremely serious. Because the software is often used to handle high-value creative assets and commercial projects, once exploited, it could result in theft or destruction of project files and potentially trigger broader data breach incidents. **In addition, since Substance 3D Painter often works in conjunction with other creative software (such as Maya, Unreal Engine, etc.), this type of vulnerability may also become a springboard for attackers to move laterally, threatening the security of the entire digital content creation pipeline.**

FIGURE 18. Comparison results for question 4.

large language model offers only a broad description of vulnerability and fails to include specific affected software versions, reflecting the degree of hallucination. In contrast, the proposed framework delivers a detailed explanation from three aspects—software overview, root cause, and vulnerability details—and includes the specific versions of the affected

Knowledge Graph Retrieval Result	**Vulnerability Name: OpenCart File Upload Vulnerability** **Submission Date: February 19, 2025** Vulnerability Description: OpenCart is an open-source e-commerce system developed by the OpenCart team in China. The system provides modules such as product reviews, product ratings, and product additions. **Version 1.0 of OpenCart has a file upload vulnerability, which is caused by a lack of effective validation for uploaded files via the image parameter. An attacker can exploit this vulnerability to upload malicious files and thereby remotely execute arbitrary code.** **Vulnerability Name: IBM Power Hardware Management Console Path Traversal Vulnerability** **Submission Date: February 19, 2025** Vulnerability Description: IBM Power Hardware Management Console (HMC) is a graphical interface software developed by IBM (International Business Machines Corporation) in the United States, used for configuring and managing the Power System series of servers. **This software is mainly used to manage hardware such as servers. IBM Power Hardware Management Console has a path traversal vulnerability, which is due to the program's failure to properly filter special elements in resource or file paths. Attackers can exploit this vulnerability to read arbitrary files.**
LLM	**As of my knowledge update date (July 2024), I am unable to directly provide information about vulnerabilities dated February 19, 2025, as future vulnerability data has not yet been publicly disclosed or may not yet exist. Vulnerability information is typically released after discovery by vendors, security organizations (such as CVE, CNVD, etc.), or researchers, and is subject to delay.**
Framework-Enhanced Generation	**Vulnerability Name: OpenCart File Upload Vulnerability** Submission Date: February 19, 2025 Vulnerability Overview: OpenCart, as a widely used open-source e-commerce system, **was revealed to have a severe file upload vulnerability in its version 1.0 on February 19, 2025. The core issue of this vulnerability lies in the system's image parameter lacking adequate security validation when handling user-uploaded files. Specifically, an attacker can craft malicious files to bypass the system's security checks and upload harmful code-containing files to the server. Once successfully uploaded, the attacker can use these files to execute arbitrary code on the remote server, thereby gaining full control over the affected system. This type of vulnerability is highly dangerous, potentially leading to data theft or tampering, and could be used to launch larger-scale cyberattacks such as Distributed Denial of Service (DDoS) or malware propagation. Therefore, OpenCart users must immediately update to the patched version or apply the official fix to avoid potential security risks.** **Vulnerability Name: IBM Power Hardware Management Console Path Traversal Vulnerability** Submission Date: February 19, 2025 Vulnerability Overview: IBM Power Hardware Management Console (HMC) is a graphical management tool developed by IBM for Power System series servers, used for configuring and monitoring hardware devices. **However, on February 19, 2025, researchers discovered a path traversal vulnerability in the software. The root cause of this vulnerability is that HMC fails to correctly handle special characters (such as "../") in file paths submitted by users, allowing attackers to access arbitrary files by constructing malicious paths. For example, attackers may read sensitive configuration files, log files, or even password files to gain unauthorized access to information. This type of vulnerability is particularly dangerous as it may be used to further infiltrate the target network or steal trade secrets. Given that HMC is typically used to manage critical business servers, this vulnerability poses a serious threat to the operational security and data privacy of enterprises. IBM subsequently issued a security advisory, recommending users to upgrade to the patched version or configure access control policies to mitigate the risk.**

FIGURE 19. Comparison results for question 5.

Question	Matched Chunk	RAG+LLM	Framework-enhanced Generation
What is Retrieval-Augmented Generation (RAG)?	**Is local AI deployment a hidden trap? RAG exposes a major security vulnerability.** Tech Update — GoUpSec, 2025-04-30 Surprisingly, RAG, which was originally intended to improve model accuracy and security, has become a sophisticated method of "jailbreaking."The more documents stored in the local knowledge base (even if they are secure), the more likely they are to trigger alignment vulnerabilities in large models and breach safety guardrails. In the rapid advancement of AI technology, Retrieval-Augmented Generation (RAG) is widely regarded as a key technology for improving the accuracy and reliability of enterprise AI. The core idea of RAG is to retrieve relevant content from external (usually local) knowledge bases to provide grounded context for large language models, thereby reducing hallucinations and improving generation quality. However, on April 28, 2025, Bloomberg published a research report revealing a shocking side effect: RAG not only fails to enhance AI security but may also render large models insecure and even lead to model "jailbreaking," posing a major new security challenge for enterprises deploying AI locally. **"The more knowledge RAG has, the more subversive it becomes"** Bloomberg's research paper on RAG security (link at the end) conducted a detailed evaluation of 11 popular large models, including Claude-3.5-Sonnet, Llama-3-8B, and GPT-4o.	Retrieval-Augmented Generation (RAG) is a technique that provides large language models with context by retrieving relevant content from external databases, thereby improving generation accuracy and reducing hallucinations.	Retrieval-Augmented Generation (RAG) is a technique that retrieves relevant information from external databases to provide context to large language models, aiming to reduce hallucinations and improve output quality.
What is the major vulnerability exposed in RAG?		The major vulnerability exposed by RAG is this: the more documents there are in a local knowledge base, the more likely they are to trigger alignment vulnerabilities in large models, breaking through safety guardrails and causing the model to "jailbreak," making the model unsafe.	The major vulnerability exposed by RAG is this: **retrieving even safe external documents as context may bypass the model's built-in safety mechanisms, resulting in unsafe responses or even triggering "jailbreaking"—even if the retrieved documents themselves are secure. Researchers have referred to this phenomenon as "the more knowledge, the more subversive", indicating that as the context length increases, the model's security may actually decline.**

FIGURE 20. Comparison results for question 1 and question 2.

Question	Matched Chunk	RAG+LLM	Framework-enhanced Generation
What is crosstalk in quantum computing?	The Dual Challenge of Crosstalk and Rowhammer Attacks Crosstalk in quantum computers refers to the phenomenon where a component (such as a qubit, control line, electromagnetic field, resonator, or photodetector) in an experimental device unintentionally affects another component. These unintended interactions may introduce errors, affect computational accuracy, and more seriously, can be exploited as security vulnerabilities. This paper investigates Rowhammer attacks in quantum computers, a type of hardware vulnerability commonly found in classical computing. In classical systems, Rowhammer attacks induce bit flips in adjacent memory locations by repeatedly accessing specific memory rows. These unwanted bit flip errors increase as the number of devices per unit area on hardware units increases. In quantum computers, similar attacks can be achieved through specific quantum gate operations. Researchers explored how crosstalk in quantum computers could be leveraged to carry out Rowhammer attacks, thereby flipping target qubits. Specifically, the team studied whether a target qubit could be flipped by applying a set of universal quantum gates. To this end, they designed experiments involving single-qubit rotations and two-qubit operations, particularly controlled-NOT (CNOT) gates. They selected multiple single-qubit gates (such as P, RZ, RX, RY, √x, √x†, Y, and U) for testing and conducted multiple experiments under different configurations to measure the output of the target qubit. The researchers analyzed the effects of single-qubit and two-qubit gate operations on the target qubit and assessed the feasibility of inducing qubit flips via crosstalk.	Crosstalk in quantum computers refers to the phenomenon where one component (such as a qubit or control line) unintentionally affects another component, potentially introducing computational errors or security vulnerabilities.	Crosstalk in quantum computers refers to the phenomenon where an experimental component (such as a qubit or control line) unintentionally affects another component. Such unintended interactions may introduce errors, degrade computational accuracy, and even be exploited as security vulnerabilities.

FIGURE 21. Comparison results for question 3.

Question	Matched Chunk	RAG+LLM	Framework-enhanced Generation
What do vulnerabilities in quantum computing primarily refer to?	"Vulnerabilities" in Quantum Computers Emerge: Classical Hacking Techniques Transcend Eras Quantum Technology · Photon Box · 2025-04-18 A research team has developed a method to infiltrate noisy quantum computers based on Rowhammer attacks. When Google's Sycamore quantum processor claimed to achieve "quantum supremacy" in 2019—completing a computation in 200 seconds that would take a classical supercomputer ten thousand years—the mystique surrounding the quantum race grew even stronger. With exponentially increasing computing power, quantum computers aim to challenge the foundations of modern cryptography. Shor's algorithm hangs like a sword of Damocles over public key cryptosystems such as RSA and ECC, and a growing sense of cryptographic crisis is spreading across global sectors like science and finance. The debate over whether "quantum computers can break the RSA-2048 encryption scheme" has become increasingly heated—but what is the real answer? The laws of quantum mechanics make it impossible to copy quantum information, but that does not mean quantum computers are immune to hacking. On March 27, a research team from the University of Gdańsk published a paper on arXiv titled "Hacking Quantum Computers with Rowhammer Attack." In the game of cybersecurity offense and defense, who can truly claim to be invincible? Research Background Quantum computing is moving toward real-world applications and is expected to deliver unprecedented processing capabilities, with potential breakthroughs in fields such as cryptography, materials science, and artificial intelligence.	The so-called "vulnerabilities" of quantum computers mainly refer to the risk of infiltrating noisy quantum systems using classical hacking techniques like the Rowhammer attack. Although quantum computers are theoretically equipped with immense computational power, the research team has demonstrated that they can still be threatened by traditional attack methods.	These vulnerabilities largely concern hardware-level attacks, such as Rowhammer attacks exploiting crosstalk effects. **This type of attack involves repeatedly executing specific quantum gates (such as CNOT gates) to interfere with neighboring qubits, causing unintentional flips in the state of target qubits—with success rates reaching up to 94.83%. This threatens the integrity and reliability of quantum computation results and reveals that quantum computers may share similar security risks with classical hardware systems.**

FIGURE 22. Comparison results for question 4.

software, thereby effectively mitigating the hallucination issue.

For Question 2 (What is the remediation for the CSRF in JFinalCMS?), the knowledge graph provides only a link to

the vendor's homepage without offering a concrete solution. The large language model presents generalized remediation suggestions. However, the proposed framework not only outlines specific remediation steps but also includes a proof of concept (PoC) tailored to the underlying principles and technical details of vulnerability.

For Question 4 (What vulnerability corresponds to CVE-2024-49515?), the knowledge graph provides the CVE identifier, software information, and a brief description. Although concise, it lacks clarity and readability. The large language model, which is trained on static data, suffers from poor timeliness and is therefore unable to return accurate information for this CVE. By contrast, the proposed framework delivers both basic details (such as the CVE ID) and an in-depth, accessible explanation of vulnerability.

Compared with traditional Retrieval-Augmented Generation (RAG) methods, knowledge-graph-based RAG offers significant advantages. Its core strength is its ability to provide large language models with richer contextual information. This capability not only enhances the model's generative performance but also effectively addresses the limitations of traditional RAG methods in understanding global or complex questions.

To validate this conclusion, we consulted domain experts in power system security and designed four representative questions for comparative experiments, as shown in Table 9. Appendix B presents the results of the comparision. A summary of these comparisons is presented in Table 10.

The experimental results indicate that, compared to traditional RAG methods, the proposed framework demonstrates superior performance in addressing global questions. For example, in Question 1 (What is Retrieval-Augmented Generation (RAG)?), which is a local question, the large language model only needs to understand a specific detail within the vulnerability description to provide an answer. In this case, the responses generated by both the traditional RAG and proposed framework are generally similar, as no broader contextual understanding is required.

However, for Question 2 (What is the major vulnerability exposed in RAG?), which is a global question, the model must comprehend the entire content of the vulnerability description to formulate an appropriate response. The traditional RAG method merely returns the name of the vulnerability, without explaining its cause. In contrast, the proposed framework builds upon the traditional RAG output by providing a comprehensive explanation and summary of the root cause of vulnerability, demonstrating its enhanced capability in handling complex, context-dependent queries.

VI. CONCLUSION

This paper proposes an RAG framework based on a knowledge graph of cybersecurity vulnerabilities in power networks. By enhancing a general LLM through this framework, the efficiency of solving power network security issues can be significantly improved. The results of four experiments demonstrate that the proposed framework significantly improves model performance across multiple dimensions. First, it outperforms existing RAG methods on five key metrics: Answer Relevancy reached 92.41%, Faithfulness 91.06%, Context Precision 98.17%, Context Recall 97.28%, and Answer Similarity 93.39%. Second, in practical application tests, the framework achieved a vulnerability verification rate of 84% and a repair rate of 92%, confirming its feasibility. Third, ablation studies validated the effectiveness of each component, as removing any module led to notable declines in retrieval and generation performance. Fourth, qualitative analysis shows that the framework effectively mitigates hallucinations, enhances timeliness and interpretability, strengthens the ability to address global issues, and improves overall generation quality.

In addition, for knowledge graph construction and intent recognition, this study adopts a prompt-based approach that leverages LLMs within the framework, which effectively improves processing efficiency. By integrating knowledge graphs with LLMs, this study provides an effective method for the extraction and utilization of power network security information. It contributes to addressing real-world challenges in power network security and lays the foundation for extension and application in other specialized fields.

However, this study has several limitations. First, in terms of knowledge graph construction, entity disambiguation and alignment have not yet been implemented, which affects the completeness and accuracy of the knowledge graph to some extent. This limitation may lead to semantic ambiguities during the question-answer generation process, thereby affecting the quality of the generated responses. Second, the current approach still requires improvements in handling complex security threats. For example, deriving attack paths based on an existing knowledge graph to enable an automated response and efficient mitigation remains a challenge. It is necessary to further integrate reasoning mechanisms between the knowledge graph and LLMs to enhance the cybersecurity defense capabilities of the system. Third, when generating vulnerability POCs, the inherent context limitations of large language models may cause the framework to fail if the generated POC exceeds the context length. To address this issue, future work could introduce a multi-agent mechanism, where different tasks are distributed to specialized agents. For example, a dedicated agent could handle vulnerability verification, effectively mitigating the problem. Finally, current comparative experiments lack comprehensive, multi-dimensional human evaluations. In the future, incorporating assessments from domain experts and end users could further improve the reliability and practical value of the model output.

APPENDIX A
QUALITATIVE ASSESSMENT 1 RESULTS

The results of the qualitative assessment 1 are shown in Figure 15, Figure 16, Figure 17, Figure 18, and Figure 19.

APPENDIX B
QUALITATIVE ASSESSMENT 2 RESULTS

The results of the qualitative assessment 2 are shown in Figure 20, Figure 21, and Figure 22.

REFERENCES

[1] Y. Yang, J. Wu, Z. Gao, Z. Liang, C. Hong, P. Li, and Y. Zhang, "Wargaming technology towards cybersecurity threat evolution in power information network," *Southern Power Syst. Technol.*, pp. 1–11.
[2] Y. Yao, J. Duan, K. Xu, Y. Cai, Z. Sun, and Y. Zhang, "A survey on large language model (LLM) security and privacy: The good, the bad, and the ugly," *High-Confidence Comput.*, vol. 4, no. 2, Jun. 2024, Art. no. 100211.
[3] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-T. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 9459–9474.
[4] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, D. Metropolitansky, R. Osazuwa Ness, and J. Larson, "From local to global: A graph RAG approach to query-focused summarization," 2024, *arXiv:2404.16130*.
[5] Z. Cheng, X. Chen, J. Wang, Z. Zhou, and Z. Zhang, "Retrieval-augmented generative intelligence question answering technology based on knowledge graph," *Comput. Sci.*, vol. 52, no. 1, pp. 87–93, 2025.
[6] W. Y. Liang, L. S. Yi, and X. Yong, "Apply knowledge map technology to improve the security level of power data and network information," in *Proc. Panda Forum Power Energy (PandaFPE)*, Apr. 2023, pp. 1121–1125.
[7] S. Qiu, Z. Shao, J. Wang, S. Xu, and J. Fei, "Research on power cyber-physical cross-domain attack paths based on graph knowledge," *Appl. Sci.*, vol. 14, no. 14, p. 6189, Jul. 2024.
[8] Y. Sun, Z. Duo, Z. Jie, and H. Wang, "A temporal knowledge graph application for network security of power monitoring system based on KNN and SVM," in *Proc. IEEE 10th Int. Conf. Inf., Commun. Netw. (ICICN)*, Aug. 2022, pp. 292–296.
[9] J. Devlin, M. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. Conf. North Amer. chapter Assoc. Comput. Linguistics, Hum. Lang. Technol.*, 2018, pp. 4171–4186.
[10] T. B. Brown et al., "Language models are few-shot learners," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 1877–1901.
[11] J. Achiam et al., "GPT-4 technical report," 2023, *arXiv:2303.08774*.
[12] X. Bi et al., "DeepSeek LLM: Scaling open-source language models with longtermism," 2024, *arXiv:2401.02954*.
[13] A. Liu et al., "DeepSeek-v2: A strong, economical, and efficient mixture-of-experts language model," 2024, *arXiv:2405.04434*.
[14] D. Guo et al., "DeepSeek-r1: Incentivizing reasoning capability in LLMs via reinforcement learning," 2025, *arXiv:2501.12948*.
[15] A. Liu et al., "DeepSeek-V3 technical report," 2024, *arXiv:2412.19437*.
[16] A. Zaboli, S. L. Choi, T. Song, and J. Hong, "ChatGPT and other large language models for cybersecurity of smart grid applications," in *Proc. IEEE Power Energy Soc. Gen. Meeting (PESGM)*, Aug. 2023, pp. 1–5.
[17] N. Ibrahim and R. Kashef, "Exploring the emerging role of large language models in smart grid cybersecurity: A survey of attacks, detection mechanisms, and mitigation strategies," *Frontiers Energy Res.*, vol. 13, Mar. 2025, Art. no. 1531655.
[18] G. Dalzell, C. Liu, S. Shah, E. Kranz, X. Yu, and M. Almashor, "Enhancing SCADA alarm management for power grids using large language models," in *Proc. IECON - 50th Annu. Conf. IEEE Ind. Electron. Soc.*, Nov. 2024, pp. 1–6.
[19] X. Luo and Y. Chen, "Knowledge filtering and aggregation driven method for retrieval-augmented generation," *J. Chin. Comput. Syst.*, vol. 2024, pp. 1–9, Sep. 2025.
[20] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang, "Retrieval-augmented generation for large language models: A survey," 2023, *arXiv:2312.10997*.
[21] R. Nakano, J. Hilton, S. Balaji, J. Wu, L. Ouyang, C. Kim, C. Hesse, S. Jain, V. Kosaraju, W. Saunders, X. Jiang, K. Cobbe, T. Eloundou, G. Krueger, K. Button, M. Knight, B. Chess, and J. Schulman, "WebGPT: Browser-assisted question-answering with human feedback," 2021, *arXiv:2112.09332*.
[22] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to retrieve, generate, and critique through self-reflection," in *Proc. 12th Int. Conf. Learn. Represent.*, 2023, pp. 1–8.
[23] Z. Rackauckas, "RAG-fusion: A new take on retrieval-augmented generation," 2024, *arXiv:2402.03367*.
[24] T. Yu, S. Zhang, and Y. Feng, "Auto-RAG: Autonomous retrieval-augmented generation for large language models," 2024, *arXiv:2411.19443*.
[25] Z. Wei, W.-L. Chen, and Y. Meng, "InstructRAG: Instructing retrieval-augmented generation via self-synthesized rationales," in *Proc. 13th Int. Conf. Learn. Represent. (ICLR)*, 2025, pp. 1–11.
[26] Z. Li et al., "Liability judgment method of telecom customer complaint based on linked-RAG and large language models," *J. Beijing Univ. Posts Telecommun.*, vol. 2025, pp. 1–6, Sep. 2025.
[27] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "RAGAs: Automated evaluation of retrieval augmented generation," in *Proc. 18th Conf. Eur. Chapter Assoc. Comput. Linguistics, Syst. Demonstrations*, 2024, pp. 150–158.
[28] Z. Wang et al., "Research on industrial question answering based on multi-agent collaborative graph RAG method," *Comput. Eng.*, vol. 2025, pp. 1–10, Sep. 2025.



XINGZHENG GAO received the bachelor's degree in computer science and technology from Liaoning Technical University, China, in 2023. He is currently pursuing the master's degree with the University of Chinese Academy of Sciences (UCAS). His research interests include power information security and cyber attack and defense.



XING CHANG received the master's degree in computer software and theory from Liaoning University, China, in 2012. He is currently an Engineer with Shenyang Institute of Computing Technology. His research interests include security penetration and electric power information security.